

Ticket Management Platform

Consolidated Business & Functional Specification
Baseline for a Long-Term, Reliable and Configurable Solution

Document Status: Business/functional baseline — database, API, UI and implementation design should follow this specification.

Core Design Philosophy: Build a stable core that rarely changes, while allowing organizational structures, roles, capabilities, ticket types, workflows, service arrangements, SLAs, notifications and integrations to evolve through controlled configuration.

1. Purpose and Objectives

- The platform is intended to provide a secure, reliable and auditable way to generate, classify, assign, coordinate, work on, validate and close tickets/complaints/requests.
- It must support dynamic organizations with different operational models, including organizations that manage tickets themselves and organizations that purchase managed ticket-operation services.
- The system should remain useful for the long term. Future business requirements and innovations should be accommodated through configuration and extension points wherever practical, without major changes to the stable backend core.
- The objective is not to reproduce any existing product; it is to establish a clean foundation that prioritizes reliability, simplicity, security, auditability and future adaptability.

2. Organizational Structure

- **SUPER USER → PROFIT CENTRE → COST CENTRE / ORGANIZATION**
- **Super User:** The highest-level platform authority. The Super User uses Profit Centres as a distributed way to navigate and oversee the application rather than treating every organization as one undifferentiated list.
- **Profit Centre:** The key distribution, management and navigation layer for the Super User. A Profit Centre contains the Cost Centres/Organizations assigned to it.
- **Cost Centre / Organization:** For this ticket management system, Cost Centre and Organization effectively represent the operational level. The system does not need additional unnecessary hierarchy below it.
- **Important:** Profit Centre is primarily a management/distribution concept for the Super User, while the Organization/Cost Centre is the actual operational boundary.

3. Data Visibility and Organizational Scope

- The Super User can see Profit Centres and drill into their Cost Centres/Organizations.
- An Organization/Cost Centre can see all of its own authorized tickets, users, activities, reports and history.
- One Organization must not automatically see another Organization's data.
- Visibility is a security rule, not merely a frontend menu rule. Backend authorization must enforce organizational scope.

4. Self-Managed and Provider-Managed Organizations

- Organizations may operate their ticket management themselves through their own authorized administrators and users.
- Some organizations may purchase a service in which the platform provider operates ticket activities on their behalf because their own administration is not regularly performed.

- In a managed arrangement, the Super User can assign an authorized provider employee to perform ticket-management work for that Organization on a paid/service basis.
- The assigned provider employee does not become the owner of the Organization. Ownership, operational responsibility and user authority remain separate concepts.
- This allows the same platform to support both self-managed and managed-service customers.

5. Users, Roles, Permissions and Capabilities

- Roles are a key foundation, but a role should not be treated as a permanent list of hard-coded actions.
- The preferred model is **User → Role → Granted Capabilities → Permitted Actions**.
- Capabilities may include View Ticket, Create Ticket, Assign Ticket, Reassign Ticket, Add Comment, Request Information, Perform Work, Send for UAT, Perform UAT, Close Ticket, Reopen Ticket and View Reports.
- Front-end roles/capabilities may vary according to business requirements. This flexibility must not expose the backend.
- **Rights Revocation:** The Super User has the authority to revoke rights, while an Admin may revoke only rights within their authorized organizational scope; neither can bypass security and audit controls.

6. Front-End and Backend Security

- Frontend role and visibility determine what the user sees and requests; they do not constitute backend security.
- The backend independently validates authentication, authorization, organizational scope, business rules and workflow rules.
- **Security flow:** User → Frontend → Authentication → Authorization → Business Validation → Workflow Validation → Backend Service → Database.
- Ordinary users must not receive direct database access, backend credentials, internal administrative access or unrestricted internal API access.
- Hiding a button is not a security control. An unauthorized user attempting an API operation must still be rejected by the backend.

7. Ticket Management

- The ticket is the central operational object representing a complaint, request, incident, development task or other supported service interaction.
- A ticket should conceptually retain requester, Organization, ticket type, category, tags, priority, assignment, team/queue where applicable, current workflow state, description, comments, work information, attachments, UAT information where applicable, resolution and closure information.
- Assignment and reassignment are controlled operations and should be auditable.

8. Ticket Types, Categories and Tags

- Ticket types should be configurable rather than permanently hard-coded.
- Examples may include Complaint, Incident, Service Request, Development Request, Change Request and Information Request.
- Organizations may require additional tags or classifications. The platform should allow controlled addition and assignment of such classifications.
- Different ticket types may use different workflows and validation requirements.

9. General Ticket Lifecycle

- **Ticket Generated → Assigned → Coordination Required from Complainant → Development / Work → UAT if Necessary → UAT Successful → Closed**
- This is a generalized business lifecycle rather than a mandatory identical sequence for every ticket.
- A simple ticket may follow Generated → Assigned → Work → Closed.
- A development request may require Coordination and UAT.
- UAT is therefore conditional: it is used where necessary and is not forced on tickets for which it is irrelevant.

10. Workflow Engine

- The workflow must be configurable and should not depend on hard-coded screen logic for every possible business path.
- **Current State → Allowed Transition → Conditions → Validation → Next State**
- The engine determines whether a transition is allowed, who may perform it, what information is required and whether the next stage is applicable.
- This allows future workflow variations to be introduced without major changes to the core ticket architecture.

11. User-Facing Status Aggregation

- The internal workflow may contain detailed operational states, but users should not be forced to understand every internal state.
- For example, internal states may include Assigned, Waiting for Complainant, Information Received, Analysis, Work Started, Work Completed, UAT Requested, UAT In Progress and UAT Passed.
- A normal user may see an aggregated status such as **In Progress**, while an operational user may see **Waiting for UAT**.
- **Principle:** One internal workflow can provide different user-facing views according to responsibility and need.

12. Validation Rules

- Validation should be considered at application, business, authorization and workflow levels.
- For example, after an Admin assigns a ticket to a user, the user must perform the required work and provide the required information before the ticket can be closed.
- The system should validate assignment, required work, resolution details, coordination requirements, UAT applicability, UAT outcome, mandatory fields/attachments and current-user authority before closure.
- No user should be able to bypass a required workflow stage simply by selecting a final status.

13. User Acceptance Testing (UAT)

- UAT is conditional and should be required only when the applicable ticket type/workflow requires it.
- **Development / Work → UAT → UAT Successful → Closed**
- If UAT fails, the ticket returns to the appropriate work stage while preserving the UAT history.
- Tickets for which UAT is not relevant can follow a shorter workflow.

14. Closed Ticket Immutability and Reopening

- **Once a ticket is closed, its historical record is immutable.**
- Users with appropriate visibility may review the complete ticket history, but previous closed activities must not be edited or silently rewritten, even by an administrator.
- An authorized Admin may reopen a ticket, but reopening must not unlock the old history.

- Reopening creates a new activity cycle: Original Work → Original Closure → Reopened → New Work → New UAT if required → Closed Again.
- **Principle:** Administrative authority can control workflow but cannot rewrite history.

15. Audit Trail

- Important operations must be auditable, including who performed the action, what happened, when it occurred, previous state, new state and reason/comment where applicable.
- Audit history should cover ticket creation, assignment, reassignment, comments, work activities, attachments, UAT, reopening, closure and administrative actions.
- The audit record must remain protected from unauthorized modification so that the ticket history remains trustworthy.

16. Teams, Queues and Assignment

- Tickets may be assigned directly to a user or initially to a team/queue depending on the organization's operating model.
- Teams/queues support operational distribution and can later enable workload balancing, skill-based routing and escalation.
- Team assignment does not replace organizational ownership; it is an operational routing mechanism.

17. Service Management and Paid Operational Support

- The platform should support service arrangements where the provider performs ticket-management activities for an Organization for a fee.
- **Organization → Service Arrangement → Assigned Service Personnel → Granted Capabilities**
- Potential future models include managed ticket operations, dedicated coordinator, dedicated technical resource, SLA-based support, monthly service and per-ticket service.
- Commercial/service concepts should remain separate from the core ticket ownership model so that they can evolve independently.

18. SLA and Escalation

- SLA policies should be configurable by Organization, ticket type, priority and/or service arrangement.
- Possible controls include response target, resolution target, warning threshold, breach detection, escalation level, business calendar, holidays and time zones.
- **Priority → Response Target → Resolution Target → Warning → Escalation**
- SLA processing should be engine-driven rather than dependent on users remembering to monitor deadlines manually.

19. Notifications

- Notifications should be implemented as an event-driven capability rather than hard-coded separately into every ticket operation.
- **Event → Notification Rule → Recipient → Channel**
- Possible events include creation, assignment, information request, UAT request/completion, closure, SLA warning and SLA breach.
- Channels may include email, in-application notifications and future messaging/push integrations.

20. Configuration and Versioning

- Important business configurations such as workflows, ticket types, permissions, SLA rules and notification rules should support controlled configuration and, where necessary, versioning.
- Existing tickets must retain the workflow/configuration context applicable to their historical processing.
- Future configuration changes should not unexpectedly alter the interpretation or processing history of existing tickets.

21. Integration Architecture

- The ticketing core should remain independent of external systems.
- Potential integrations include email, messaging, ERP, CRM, monitoring systems, core banking systems, external APIs, reporting systems and AI services.
- **Ticketing Core → Integration Layer → External Systems**
- External systems should interact through controlled interfaces rather than directly manipulating core ticket tables.

22. Reporting and Dashboards

- Reporting must follow the same organizational scope and authorization rules as operational functionality.
- The Super User can aggregate across the platform, a Profit Centre can aggregate across its Organizations, and an Organization can report within its own authorized scope.
- Possible metrics include open tickets, aging, SLA performance, workload, UAT queue, closure trends, escalations and service performance.

23. Global and Multi-Organization Readiness

- The architecture should support multiple independent Organizations without allowing data leakage between them.
- Global readiness should consider time zones, business calendars, holidays, localization and organization-specific operating rules.
- Organizational scope should be a first-class security concept from the beginning rather than a feature added later.

24. Retention, Archival and Recovery

- Closed tickets may eventually be archived according to configurable retention policies while remaining available for authorized historical review.
- Backup and disaster recovery must preserve operational data and audit history.
- Archival must not be treated as permission to rewrite or improperly delete required historical records.

25. Future Extensibility

- The system should provide extension points for new ticket types, workflow stages, permissions/capabilities, SLA models, notification channels, service arrangements and integrations.
- Future innovation should preferably be introduced by adding configuration or isolated services rather than changing stable core tables and business logic unnecessarily.
- The design must balance flexibility with control: not everything should be configurable, and security-critical rules must remain protected by the core platform.

26. Architectural Philosophy

- **Stable Core:** Fundamental ticket, security and audit architecture should change rarely.
- **Configuration Over Hard-Coding:** Business variations should be configurable wherever practical.
- **Simple User Experience:** Users see only what they need for their responsibility.

- **Strong Backend Security:** Frontend roles and permissions never become direct backend privileges.
- **Hierarchical Visibility:** Super User uses Profit Centres for distributed oversight; Organizations operate within their own scope.
- **Capability-Based Authority:** Roles remain flexible and capabilities provide finer control.
- **Workflow Control:** Required business stages cannot be bypassed.
- **Immutable History:** Closed work cannot be rewritten.
- **Reopen, Don't Rewrite:** Reopening creates new activity while preserving old history.
- **Future Extension:** New requirements should be accommodated without major core redesign where practical.
- **Central Principle:** Complexity belongs in the engine; simplicity belongs in the user experience.

27. Final Consolidated Concept

- **SUPER USER → PROFIT CENTRE → COST CENTRE / ORGANIZATION**
- The Super User uses Profit Centres to manage the platform in a distributed manner. Each Organization/Cost Centre is the operational boundary for its own users and tickets.
- An Organization may be self-managed or may use provider-managed ticket operations through assigned service personnel with controlled capabilities.
- Tickets follow configurable workflows. Roles and capabilities determine who may perform actions; backend authorization and validation remain mandatory.
- UAT is used only where applicable. Closure requires completion of the required workflow. Once closed, the historical ticket record is immutable; reopening creates new activity rather than rewriting the past.
- SLA, escalation, notifications, reporting, integrations and future service models should be configurable and extensible.
- **Final Design Goal:** Create a long-lasting platform with a stable, secure core and a flexible outer layer, so that future innovation can be introduced simply without requiring major changes to the backend architecture.

Architecture at a Glance

Layer	Primary Responsibility
Organizational Model	Super User → Profit Centre → Cost Centre / Organization
Ownership & Service	Self-managed or provider-managed Organization
Identity & Authority	Users → Roles → Capabilities → Permissions
Ticket Engine	Ticket types, categories, tags, assignment, teams/queues
Workflow Engine	States, transitions, conditions and workflow versions
Validation	Application + Business + Authorization + Workflow validation
Operational Quality	UAT, SLA and escalation
Communication	Event-driven notifications
Audit	Immutable historical activity and closure records
Integration	Controlled APIs/integration layer
Future Extension	Configuration and isolated extension points

Baseline rule: Database schema, APIs, frontend screens and implementation should be derived from this business/functional specification, not the other way around.